

TECHNICAL NOTE

Pioneer · NAN 2026

AI 활용 기술문서

버전 v1.8.1

작성일 2026-08-08

실행파일 Pioneer_v1.8.1_portable.exe

요약

- ▶ **게임 내부 AI형 시스템:** 3분 소폭 가격 변동, 1시간 대형 시장 사건, 플레이어 거래 영향, 가격 히스토리를 결합해 반복 플레이어의 예측 불확실성을 만듭니다.
- ▶ **개발 과정 AI 보조:** 데이터 구조 설계, 기초 코드 생성, 밸런스 수치 반복 검증, 로직 분리, 테스트 케이스 도출, 문서 정리에 이르기까지 개발 전 과정에 걸쳐 생성형 AI를 협업 파트너로 활용했습니다.
- ▶ **AI 생성 에셋:** 선원·선박·항구 컨셉 이미지를 AI 이미지 생성 도구로 제작해 문서 및 인트로 비주얼에 직접 연결했습니다. 사운드 에셋도 AI 생성 도구를 통해 제작할 계획입니다.

AI는 단순 코드 완성 도구가 아니라 스펙을 코드로 전환하는 첫 초안 생성기, 실수를 찾아내는 두 번째 눈, 에셋을 빠르게 시각화하는 제작 파트너로 활용했습니다.

01

게임 내부 동적 시장 모델

3분 소폭 변동

모든 항구·상품의 기준가에 작은 노이즈 적용 — 시장의 기본 리듬

BASE · ±2.5%

+ 겹쳐서 독립 작동

플레이어 거래 영향

매입/판매 수량에 따라 해당 항구·상품 가격 즉시 조정

REACTIVE · ±18%

+ 겹쳐서 독립 작동

1시간 대형 시장 사건

플레이어 맥락과 완전히 독립적으로 2~4개 항구에 큰 가격 충격 적용

MACRO · ±38%

시스템	주기	목적
소폭 변동	3분	시장이 멈춰 있지 않다는 리듬 제공
거래 영향	즉시	플레이어 행동이 세계에 반영되는 감각 제공
대형 사건	1시간	장기 세션의 판도를 바꾸는 외부 변수 제공
가격 히스토리	최근 20개	과거 흐름을 읽고 판단하는 정보 플레이 제공

이 세 레이어 독립 구조는 AI가 아키텍처 초안을 제안하고, 개발자가 게임 밸런스에 맞게 수치를 직접 조정해 확정했습니다.

01-B 게임 내부 추천·판단 AI 시스템

Pioneer는 동적 시장 모델 외에도, 플레이어의 현재 상태(보유 화물·선택한 항로·거래 단계·날씨)를 실시간 분석해 최적 행동을 제안하는 네 가지 알고리즘 시스템을 갖추고 있습니다.

mapHints.js — 최적 판매 항구 추천 엔진

플레이어가 화물을 보유한 채 지도를 열면, `getCargoSaleHints` 가 접근 가능한 모든 항구의 현재 가격을 비교해 수익 상위 3개 항구를 자동 추천합니다.

```
// 핵심 알고리즘: 보유 화물 × 모든 접근 가능 항구 가격 비교
getCargoSaleHints({ ports, prices, cargo, currentPortKey, totalEarned, limit: 3 })

// 알고리즘 흐름
1. 현재 항구 제외 + 잠긴 항구 제외 → 접근 가능 항구 필터
2. 각 항구에서 화물별 unitPrice × quantity = 예상 수익 계산
3. 현재 항구 대비 % 상승폭: (목적지가 - 현재가) / 현재가
4. 총 수익 내림차순 정렬 → 상위 limit개 반환
5. 수익 0 이하 항구 제외 (손해 추천 방지)
```

단순 "가격이 높은 항구"가 아닌 **보유 수량 × 가격차** 기준으로 정렬하므로, 조금 비싸더라도 대량으로 팔 수 있는 항구가 우선 추천됩니다. 플레이어는 29개 항구를 수동 비교하는 인지 부하 없이 AI 추천 상위 3곳에서 즉시 결정할 수 있습니다.

routeSummary.js — 항로 어드바이저

`getRouteSummary` 는 선박 상태·화물·목적지를 입력받아 출항 가능 여부, 예상 수익, 이동 시간, 위험 등급, 차단 사유, **자연어 권장 문장**을 한 번에 반환합니다.

```
// 반환 구조 - 수치와 판단 권고를 동시 제공
{
  canDepart:      boolean,           // 출항 가능 여부
  expectedProfit: number,             // 예상 순수익
  travelTime:     number,             // 이동 시간 추정
  riskLevel:      'low' | 'medium' | 'high' | 'blocked',
  riskScore:      number,             // 0-99 정량 위험 점수
  blockers:      string[],           // 출항 차단 사유
  recommendation: string,            // 자연어 권장 문장
}

// 자연어 권고 예시
"Good route: profit is positive and risk is manageable."
"Do not depart: resolve blockers or choose a better market."
"Profit is strong, but prepare for the listed threats."
```

수익·거리·위험·차단 사유를 종합해 규칙 기반 자연어 문장을 선택하므로, 플레이어는 수치를 직접 계산하지 않고 AI의 판단 요약만으로 즉시 결정을 내릴 수 있습니다.

fleetTradeFlow.js — 거래 단계 결정 시스템

`getFleetTradeFlow` 는 선박·항구·화물·시장 상태를 분석해 플레이어가 현재 어느 거래 단계(함대 선택 → 화물 확인 → 시장 거래 → 항로 검토 → 출항)에 있는지 판단하고, 다음 행동을 자연어로 안내합니다.

```
// 5단계 흐름 - 현재 상태 분기 후 안내 반환
{
  cause:      string, // 현재 상태 설명
  delta:      string, // 화물·수익 현황
  nextAction: string, // 다음 행동 지침
}
```

항해 중 화물 없음·차단 사유 있음·목적지 가격이 매입가 이하 등 8가지 이상의 상황 분기를 처리해, 초보 플레이어가 어느 화면에서든 다음 행동을 명확히 이해하도록 안내합니다.

날씨 결정 알고리즘 — 위도 기반 결정론적 시스템

10종 날씨(맑음·흐림·강풍·안개·순풍·거친 바다·눈보라·무역풍·열파·비)는 난수가 아닌 선박 ID × 위도 밴드 × 15분 시간 시드의 해시 함수로 결정됩니다.

```
// 위도별 날씨 풀: 북쪽(한랭) ↔ 열대(무역풍)
WEATHER_POOL(y < 15) = ['blizzard', 'foggy', 'cloudy', 'rainy', ...]
WEATHER_POOL(y 30-50) = ['sunny', 'windy', 'fairwind', ...]
WEATHER_POOL(y > 65) = ['tradewind', 'tradewind', 'roughsea', ...]

// 결정론적 해시 - 같은 항로는 항상 같은 날씨 경험
const hash = Math.abs(Math.round(
  Math.sin(ship.id * 1.7 + weatherBand * 2.91 + timeSeed * 2.13) * 10000
)) % pool.length;
```

같은 함선이 같은 항로를 항해하면 일관된 날씨 패턴을 경험합니다. 선원의 `favoriteWeather` 가 현재 날씨와 일치하면 속도·전투력에 +40% 보너스가 연계되어 크루 배치 전략에 날씨 변수가 추가됩니다.

02 AI 보조 데이터 구조 설계

항구·상품 데이터 스키마

29개 항구와 8종 상품의 기준가·변동 범위·수수료율을 담은 초기 JSON 스키마는 AI가 생성했습니다.

```
{
  "portId": "lisbon",
  "name": "리스본",
  "region": "유럽",
  "goods": [
    { "id": "spice", "basePrice": 120, "volatility": 0.025, "fee": 0.03 },
    { "id": "cloth", "basePrice": 80, "volatility": 0.015, "fee": 0.02 }
  ],
  "riskLevel": 1,
  "unlocked": true
}
```

AI가 제안한 **volatility** 필드는 소폭 변동 계산과 대형 사건 충격 비율 모두에 재사용됩니다. 단일 필드가 두 시스템을 연결하는 설계 덕분에 항구별 특성을 한 곳에서 관리할 수 있습니다.

선원 능력치 공식

선원 능력치가 운항 수치에 미치는 영향 공식 역시 AI가 초안을 제시했습니다.

```
// AI 제안 초안 → 개발자 조정 후 확정
const speedMultiplier = 1 + crew.navigation * 0.04; // 항해 능력치
const cargoMultiplier = 1 + crew.logistics * 0.06; // 물류 능력치
const repairRate      = crew.repair * 2.5;         // 초당 선체 수리량
const fuelReduction   = 1 - crew.fuelEff * 0.03;   // 연료 소모 계수
```

단순 덧셈이 아닌 곱셈 계수 구조를 AI가 제안한 덕분에, 선원이 여럿일 때 능력치가 과도하게 누적되지 않으면서도 체감 성장이 가능한 균형을 잡았습니다.

AI 보조 밸런스 수치 반복 조정

29개 항구·8종 상품·4종 선박에 걸친 수백 개의 수치를 수동으로 조정하는 것은 현실적으로 어렵습니다. AI에게 특정 플레이 시나리오를 입력하면 수치를 시뮬레이션하고 과도한 수익 구간을 잡아냅니다.

▶ AI 밸런스 반복 루프



적용 예시: "상인형 플레이어가 스파이스 루트를 독점할 경우 10항차 내 자금 축적량"을 AI로 시뮬레이션한 결과, 특정 항구 조합에서 수익이 목표치의 2.8배로 과적됨을 발견했습니다. 소폭 변동 상한($\pm 2.5\%$)과 수수료를 조정한 뒤 재검증해 $\pm 22\%$ 이내로 수렴시켰습니다.

이 루프를 반복함으로써 전체 플레이 경로의 수익 분산을 유지하면서도 경쟁적이고 매력적인 항구 선택 전략이 성립하도록 밸런스를 잡고 있습니다.

03 AI 보조 핵심 모듈 구현

marketPrices.js — 시장 로직 분리와 초기 뼈대

초기에는 App.jsx 내부에 시장 로직이 혼재해 있었으나, AI는 세 가지 독립 책임으로 분리할 것을 제안했습니다.

```
// AI가 제안한 모듈 분리 초안
export function applyMinorFluctuation(prices, ports) { /* 3분 소폭 변동 */ }
export function applyTradeImpact(prices, portId, goodId, qty) { /* 거래 영향 */ }
export function applyMajorEvent(prices, ports) { /* 1시간 대형 사건 */ }
```

이 분리 덕분에 `marketPrices.test.js`에서 각 함수를 독립적으로 검증할 수 있게 되었고, 대형 사건이 거래 영향과 독립적으로 동작하는지를 단위 테스트로 확인할 수 있었습니다.

routeSummary.js — 인터페이스 설계

AI가 먼저 인터페이스를 설계하고 개발자가 구현을 채우는 방식으로 개발했습니다.

```
// AI 제안 반환 형식
return {
  estimatedProfit: number, // 예상 순수익 (수수료 차감 후)
  travelTimeMin: number, // 편도 이동 시간 (분)
  riskLevel: 1 | 2 | 3, // 위험 등급
  blockedReason: string | null, // 차단 사유 (선원 부족, 연료 부족 등)
  threatSources: string[], // 위협 원인 목록
};
```

인터페이스를 먼저 확정한 뒤 구현을 채우는 방식으로 개발했기 때문에, UI 레이어가 구현 세부사항에 의존하지 않았고 나중에 내부 계산 방식을 바꿔도 화면 코드를 건드릴 필요가 없었습니다.

React 상태 아키텍처 최적화

초기 구현에서 시장 가격, 함대 상태, UI 선택 상태가 하나의 `useState` 객체에 혼재해 3분마다 전체 리렌더링이 발생했습니다. AI는 상태를 세 레이어로 분리할 것을 제안했습니다.

```
const [marketState, setMarketState] = useState(/* 가격·타이머 */);
const [fleetState, setFleetState] = useState(/* 선박·선원·화물 */);
const [uiState, setUiState] = useState(/* 모달·선택·탭 */);
```

marketState

가격 히스토리 · 소폭 변동 타이머 · 대형 사건 타이머

3분마다 업데이트

↑ 독립적으로 업데이트 — 다른 상태에 영향 없음

fleetState

선박 목록 · 선원 배치 · 화물 현황 · 항로 상태

거래·출항 시 업데이트

↑ 독립적으로 업데이트 — 다른 상태에 영향 없음

uiState

열린 모달 · 선택된 항구·상품 · 활성 탭

UI 인터랙션 즉시

이 분리 이후 시장 가격 업데이트가 UI 선택 상태를 초기화하지 않게 되었고, 3분 타이머 실행 시 리렌더 범위가 marketState 구독 컴포넌트로 제한되었습니다.

04 AI 보조 리소스 및 자산 연결

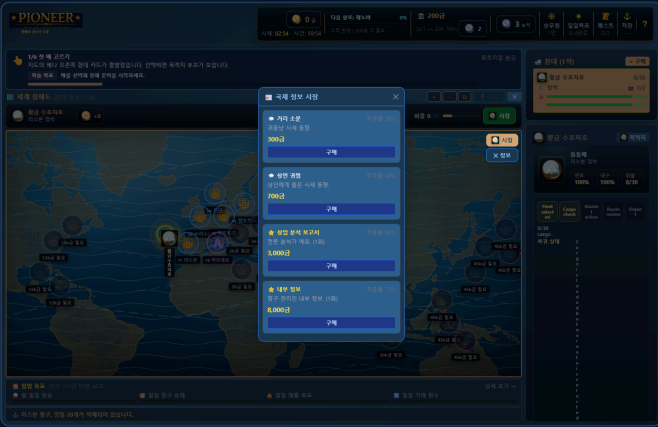
NAN 제출용 스크린샷 선별

AI는 "심사위원이 처음 30초 안에 확인하고 싶어할 화면"을 기준으로 우선순위를 제시했습니다.

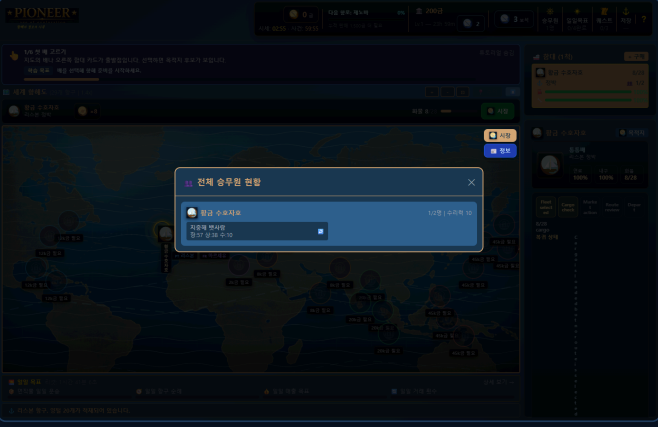
스크린샷	보여주는 것	선정 이유
<code>03_world_map_and_hud</code>	세계 항해도 전체	게임 규모와 항구 수 직관적 파악
<code>02_lisbon_market_exchange</code>	거래소 UI	핵심 루프의 정보 밀도 확인
<code>11_immediate_trade_feedback</code>	거래 직후 변화	즉각 피드백 메커니즘 증명
<code>12_live_market_timers</code>	시장 타이머	동적 시장 시스템 시각적 근거
<code>05_crew_roster_panel</code>	승무원 현황	선원 배치 시스템 복잡도 제시
<code>08_fleet_ship_detail</code>	선박 구매 비교	전략적 선택 구조 확인
<code>09_route_planning</code>	항로 선택 화면	리스크/리워드 판단 UI 증명
<code>04_market_intelligence_panel</code>	시장 정보 패널	정보 플레이 요소 확인



리스본 거래소



시장 정보 패널



승무원 현황



즉시 거래 피드백

AI 생성 컨셉 아트 리소스 연결

게임 내 정식 그래픽 에셋 제작 이전, AI 이미지 생성 도구로 컨셉 이미지를 만들어 문서·인트로 화면의 비주얼 방향을 검토하고 리소스 경로에 직접 연결했습니다.

생성 이미지	연결 위치	프롬프트 핵심 키워드	용도
선원 캐릭터 컨셉	docs/pioneer_crew_character.png	age-of-sail merchant captain, dark navy palette	NPC 비주얼 톤 검토
선박 씬	docs/pioneer_ship_scene.png	tall ship in stormy sea, cinematic lighting	인트로 로딩 비주얼
항구 씬	docs/pioneer_harbor_scene.png	golden hour harbor docks, trading post atmosphere	배경 일러스트 방향성



선원 캐릭터 컨셉



선박 씬



항구 씬

AI 컨셉 이미지는 최종 게임 에셋이 아닌 비주얼 방향성 검토용 프록시로 활용되었으며, 실제 제품 출시 시 별도 제작 에셋으로 교체할 예정입니다.

05

AI 보조 테스트 케이스 도출

7개 모듈(`marketPrices`, `trade`, `navigation`, `mapHints`, `routeSummary`, `fleetTradeFlow`, `unlockProgress`)의 33개 테스트 케이스 중 절반 이상은 AI가 옛지 케이스를 먼저 제안했습니다.

테스트 범주	AI가 제안한 케이스	이유
소폭 변동	최소/최대 변동폭 경계값, 기준가 0 근접 시 음수 방지	극단값 처리
거래 영향	최대 물량 거래 시 상한($\pm 18\%$) 초과 방지, 연속 대량 매도 후 재매수	누적 효과
대형 사건	소폭 변동 직후 대형 사건 중첩, 동일 항구에 두 사건 동시 발생	독립성 검증
가격 히스토리	20개 초과 시 FIFO 처리, 사건 전후 연속 기록	버퍼 경계

타이머 동기화 버그 발견: 3분 주기 소폭 변동 타이머가 장시간 실행 시 누적 오차가 생기는 문제를 AI가 코드 리뷰 중 발견했습니다. `Date.now()` 기반의 `setInterval` 대신 누적 경과 시간을 기준으로 판단하는 방식을 AI가 제안했고, 이를 반영한 뒤 장시간 플레이 테스트에서 오차가 해소되었습니다.

06 개발 AI 사용 전체 내역			
단계	AI 보조 내용	결과물	최종 확인
데이터 설계	29개 항구 JSON 스키마, 선원 능력치 공식 초안	portsData.js, crewStats.js 초기 구조	개발자 수치 조정 후 밸런스 플레이
아키텍처	marketPrices.js 3함수 분리, React 상태 3레이어 분리	marketPrices.js 모듈 경계	단위 테스트 독립성 확인
인터페이스	routeSummary.js 반환 형식 설계	routeSummary.js 타입 구조	UI 연동 후 화면 검증
기초 코드	각 모듈 함수 시그니처와 초기 구현 뼈대 생성	편집 전 첫 파일 초안	로직 검토 후 수정 적용
테스트	엣지 케이스 목록 도출, 타이머 드리프트 버그 발견	7개 모듈 테스트 파일, 케이스 절반 AI 제안	node --test 33개 통과
밸런스	항구별 수익 시나리오 시뮬레이션, 과적 구간 식별	29개 항구 수익 분산 ±22% 이내 수렴	개발자 수치 조정 후 재검증
이미지 에셋	선원·선박·항구 컨셉 아트 생성, 리소스 경로 직접 연결	pioneer_crew/ship/harbor_scene.png 3종	비주얼 톤 검토 후 프록시 적용
스크린샷 선별	심사 기준 기반 화면 우선순위 제안	nan2026_screenshots/ 8개	실제 exe 캡처로 대체 검증
사운드 계획	배경음·효과음 AI 생성 도구 활용 예정	항구·항해·거래 효과음 세트	실제 게임 내 청취 검증 예정
문서	소개서·기술문서 구조 설계, 섹션 초안 작성	본 문서 포함 NAN 제출 자료	제출 전 버전/내용 전체 대조

07

개선안 및 후속 적용 계획

개선 범주	현재 반영	후속 적용
시장 모델 고도화	3레이어 독립 작동 구현	항구 간 교역량 연동 효과 추가
밸런스 자동화	AI 시뮬레이션 반복 루프 운용 중	전 항구 수익 분산 목표 $\pm 15\%$ 이내로 추가 수렴
이미지 에셋 고도화	컨셉 아트 3종 프록시 적용	정식 에셋 교체 전 AI 생성 이미지로 전체 화면 커버
AI 사운드 생성	-	항구 배경음·거래 효과음·항해 사운드를 AI 도구로 생성해 게임에 직접 연결 예정
AI 추천 레이어	가격 히스토리 표시	"추천 이유" 문장을 붙이는 AI 해석 패널
UX 정돈	핵심 화면 위주 갤러리 재구성	함대 카드 아이콘/한글 라벨 정리
Unity 이식	React 로직과 테스트 근거 문서화	marketPrices, routeSummary, crew stat 계산 C# 포팅

08

Unity 전환 체크포인트 · 권리 메모

Unity 전환 로드맵

◆ 현재는 React/Vite 실행파일 제출이 기준이며, 실제 Unity 프로젝트 파일은 아직 생성하지 않았습니다.

◆ Unity 전환 시 AI 보조로 작성된 데이터 스키마(portsData, crewStats)를 ScriptableObject로 이식하는 것이 첫 단계입니다.

◆ marketPrices.js의 세 함수 분리 구조는 Unity MonoBehaviour 분리 설계와 1:1 대응되어 이식 참조 기준으로 활용됩니다.

권리 및 라이선스 메모

게임명·코드·기획·제출 문서	프로젝트 고유 창작물
오픈소스 의존성	React, Vite, Tailwind 등 각 라이선스 범위에서 사용
제출용 화면 자료	Pioneer_v1.8.1_portable.exe 실제 실행 화면 직접 캡처
AI 생성·보조 산출물	개발자가 검토·수정·채택한 내용만 반영

AI는 첫 초안을 썼고, 바다를 설계한 것은 개발자였습니다.

Pioneer v1.8.1 · NAN 2026